

Практическая работа №6

Теоретическое введение

С появлением микросервисной архитектуры стало очевидным, что традиционные методы разработки, тестирования и развертывания приложений уже не так эффективны, как раньше. Микросервисы представляют собой набор отдельных компонентов, каждый из которых может разрабатываться, тестироваться и развертываться независимо друг от друга. Это создает необходимость в автоматизации процессов сборки, тестирования и развертывания, чтобы обеспечить скорость и надежность разработки приложений.

CI/CD (Continuous Integration/Continuous Deployment) — это практика разработки программного обеспечения, которая автоматизирует процессы интеграции изменений в коде, их тестирования и развертывания в производственную среду. В контексте микросервисной архитектуры CI/CD становится основой для обеспечения непрерывной поставки изменений в каждый сервис, что позволяет командам разработки быстро и безопасно вносить изменения в приложение.

Введение CI/CD в микросервисную архитектуру позволяет:

- Ускорить процесс разработки: Автоматизация сборки, тестирования и развертывания позволяет быстро доставлять новые функции и исправления ошибок.
- Повысить качество программного обеспечения: Непрерывное тестирование обеспечивает быструю обратную связь о качестве изменений, что помогает выявлять и устранять проблемы на ранних этапах разработки.
- Снизить риск: Автоматизация процессов развертывания позволяет уменьшить риск внедрения изменений, так как они проходят через цепочку

автоматизированных тестов перед тем, как попасть в производственную среду.

- Обеспечить непрерывную поставку: CI/CD обеспечивает непрерывное развертывание изменений в производственную среду, что позволяет реагировать на потребности бизнеса быстрее и более гибко.
- Улучшить командную работу: CI/CD способствует автоматизации и стандартизации процессов разработки, что повышает эффективность работы команды и уменьшает вероятность человеческих ошибок.
- Реализация CI/CD в микросервисной архитектуре требует правильного выбора инструментов, настройки конвейеров сборки и развертывания, а также установления надежных практик тестирования и обеспечения качества кода.

Полезные ссылки

1. Официальная документация Docker: <https://docs.docker.com/>
2. Docker Hub: <https://hub.docker.com/>
3. Docker Community Forums: <https://forums.docker.com/>
4. Микросервисная архитектура: введение и основные концепции:
<https://microservices.io/>
5. Что такое CI/CD?: <https://habr.com/ru/companies/otus/articles/515078/>

Задание

Цель работы: внедрить CI/CD в уже существующий проект построенный на микросервисной архитектуре.

В данной практической работе необходимо внедрить существующий проект в Gitlab, с использованием пайплайна. Пайплайн должен содержать несколько стадий (минимум 3), к примеру такие как build, test и deploy.

В ходе выполнения практики могут быть использованы любые языковые средства и фреймворки, для создания микросервисов.

Ниже представлен пример пайплайна:

```
stages:
  - build
  - test
  - deploy

variables:
  DOCKER_HOST: tcp://docker:2375/

services:
  - docker:dind

before_script:
  - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY

build:
  stage: build
  script:
    - docker build -t $CI_REGISTRY_IMAGE:latest .
    - docker push $CI_REGISTRY_IMAGE:latest
```

test:

stage: test

image: python:latest

script:

- pip install -r requirements.txt
- python -m unittest discover tests/

deploy:

stage: deploy

image: alpine:latest

script:

- apk add --update openssh-client
- echo "\$SSH_PRIVATE_KEY" | tr -d '\r' >

~/.ssh/id_rsa

- chmod 600 ~/.ssh/id_rsa
- ssh user@server "cd /path/to/microservice && git pull && docker-compose up -d"

only:

- master

Вопросы к практической работе

1. Какие преимущества предоставляет использование CI/CD в микросервисной архитектуре?
2. Какие инструменты можно использовать для реализации CI/CD в микросервисах?
3. Каковы основные этапы процесса CI/CD в контексте микросервисов?
4. Какие проблемы могут возникнуть при внедрении CI/CD в микросервисную архитектуру и как их решить?
5. Какие тесты следует включить в цепочку CI/CD для микросервисов?
6. Какова роль контейнеризации (например, Docker) в процессе CI/CD для микросервисов?
7. Как обеспечить однородность среды разработки, тестирования и производства при использовании CI/CD в микросервисах?
8. Каким образом можно обеспечить мониторинг и логирование приложений в контексте CI/CD и микросервисов?
9. Как организовать процесс непрерывного обновления инфраструктуры для поддержки CI/CD и микросервисов?
10. Какие практики безопасности следует учитывать при реализации CI/CD в микросервисах?

Критерии оценки

- Реализовано внедрение CI/CD в проект при помощи Gitlab и пайплайна
- Проверена корректная работа проекта
- Составлен отчет о проделанной работе со скриншотами её выполнения
- Даны ответы на вопросы к практической работе